

PERFECT LABELS FOR FREE? EXACT ANSWERS TEACH SHORTCUTS; EXACT TRACES TEACH STEPS SEEN OFTEN ENOUGH

Anonymous authors

Paper under double-blind review

ABSTRACT

Exact verifiers (primality tests, SAT solvers, unit tests) label data with no noise at no marginal cost, and they supply the rewards of verifiable-reward post-training. We use them as a measurement instrument: with label noise removed, the effect of the supervision *format* can be isolated. We fine-tune Qwen2.5-1.5B-Instruct with LoRA on 180 exactly labeled examples per task and compare supervision by the answer alone with supervision by a deterministic procedure trace followed by the answer, against a scrambled-trace control. Answer-only training recovers about what simple probes on the input’s surface recover: on 4-digit primality it reaches 82.3%, and its answers coincide with the rule “the last digit is 1, 3, 7 or 9” on 99.6–100.0% of test items. On 4-digit divisibility by 7, which has no surface rule, answers stay at chance (50.3%, below the zero-shot base at 62.5%), while a 4-step long-division trace reaches 88.3% over three seeds and its scrambled counterpart 50.0%. A short correct trace is not enough: at the same length, divisibility by 13 reaches 54.6% with 2.1% of its traces correct, and on primality traces hurt (57.7%). Our preregistered prediction that trace accuracy compounds a fixed per-step accuracy over the trace length failed: 6-digit divisibility by 7 reached 96.2% against a predicted 75.8%. A successor hypothesis, preregistered before its cells ran, ties per-step accuracy to how often each (remainder, digit) transition is supervised; its first predicted cell disagrees with it, because step difficulty varies with the divisor, and its other tests are pending. The lesson concerns how to design supervision for verifiable-reward pipelines, not arithmetic.

1 INTRODUCTION

Verifiable-reward post-training rests on programs that decide correctness exactly: answer checkers for mathematics, unit tests for code, solvers for logic (Lambert et al., 2024; DeepSeek-AI et al., 2025). Such verifiers are often described as a source of perfect labels for free. They are perfect in one sense: a verified label is never wrong. Whether a model trained on those labels learns the computation the verifier performs, or a cheaper function that agrees with it on the training distribution, is a separate question, and in practice it is confounded with label noise, with task difficulty, and with the format of the supervision.

We use exact verifiers to remove the first confound. When every label is correct, any difference between two training regimes on the same inputs is a difference in what the supervision exposes. We choose small decision tasks whose structure is known in advance (which surface rules approximate the label, which steps the natural procedure takes) so that what should be learnable can be stated before training. The point is not to teach a language model what a ten-line program already computes. It is that on such tasks we can tell which function the model learned, which is what a verifiable-reward pipeline for mathematics or code needs to know about its own supervision and usually cannot measure.

We compare two supervision formats on Qwen2.5-1.5B-Instruct. *Answer-only* supervision (arm A) teaches a shortcut. On 4-digit primality the answer-trained model reaches 82.3%, between a one-hot linear probe on the digits (79.3%) and the best cross-validated surface probe (89.7%). Its answers are a one-token rule: they coincide with “the last digit is 1, 3, 7 or 9” on 99.6–100.0% of test items,

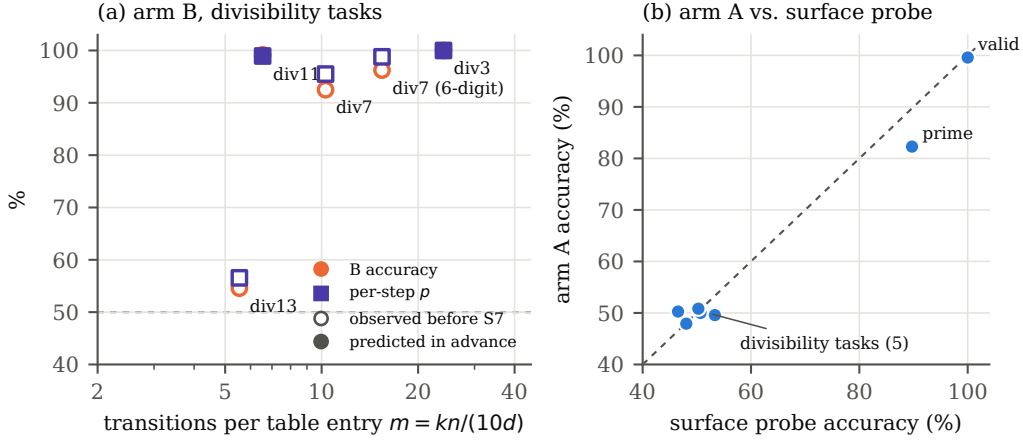


Figure 1: (a) Trace arm B on the divisibility tasks: answer accuracy and per-step accuracy p (probability that a step is right given that the previous one was) against $m = kn/(10d)$, the supervised transitions per (remainder, digit) table entry, which is known before training. Hollow: cells observed before S7 was registered; filled: cells predicted in advance by S7 or S9. (b) Answer arm A against the surface probe fitted on the same training items (one-hot features where engineered features encode the answer); the dashed line is equality.

and that rule alone scores 82.5%. On divisibility by 7, where no surface rule exists, it stays at chance. *Trace* supervision (arm B: a deterministic procedure trace, then the answer) can teach what answers cannot: the 4-step long-division trace lifts divisibility by 7 from 50.3% to 88.3% over three seeds, while a scrambled trace with the same tokens (arm C) stays at 50.0%. A correct trace is not sufficient, and neither is a short one. With the same 4 steps, divisibility by 13 barely moves (54.6% vs. 49.6%, one seed), and on primality traces hurt (57.7% vs. 82.3%).

Our preregistered account was that trace supervision works when an error-free trace is likely: with per-step accuracy p over k steps, about p^k . Its test failed (S5). Trained on 6-digit traces, the model reached 96.2% against the predicted 75.8%, because its own per-step accuracy (0.988) exceeded the 4-digit model’s (0.955). Since p is measured after training, p^k describes a trained model; it cannot say in advance which supervision will work. After the failure, and before its cells ran, we preregistered a quantity that can be computed before training (S7): the number of supervised transitions per entry of the (remainder, digit) table, $m = kn/(10d)$ for n traces of k steps modulo d . The three cells observed when we wrote it down are ordered by m (Figure 1a), but the first cell predicted in advance contradicts it: divisibility by 11 at $m = 6.5$ reached $p = 0.989$, above the predicted interval. Its steps are easier, because $10 \equiv -1 \pmod{11}$ turns each step into a subtraction. Step difficulty varies with d and confounds m ; the remaining tests of S7, a dose test at fixed d (S9) and a grounding control (S8) are reported as they come in.

Contributions.

- A controlled comparison of supervision formats under exact labels (answers, procedure traces, scrambled traces, shuffled labels) with paired tests on problem-disjoint test sets (§3).
- Evidence that exact answer labels teach the surface probe’s function rather than the verifier’s: probe-level accuracy, a last-digit rule that matches the model’s answers item by item, and failure on every composite the rule mislabels (§4.1).
- Evidence on when exact traces teach: the trace unlocks divisibility by 7 through its content, not its tokens; the same format fails for divisibility by 13 and for primality; generation-level diagnostics show where (§4.2).
- A preregistered record that includes a failed prediction (S5), a successor (S7) whose first predicted cell failed, and tests still running, all reported (Table 3).

One result reversed an earlier conclusion of ours. A verdict written on 2026-08-04 from a 60-example trace run stated that traces do not unlock divisibility by 7; the preregistered 180-example cell, which finished afterwards with the same code, gave 92.5%. We report both. Before any new run started, the replication of that cell was made a go/no-go condition for this submission, and it passed: 90.0% and 82.5% in seeds 1 and 2 (§4.2).

2 RELATED WORK

Intermediate steps as supervision. Scratchpads (Nye et al., 2021) and chain-of-thought (CoT) prompting (Wei et al., 2022) showed that emitting intermediate steps improves multi-step computation, and Sprague et al. (2025) find the benefit of CoT concentrated on mathematical and symbolic tasks. For small transformers trained from scratch, Lee et al. (2024) show that the format of arithmetic data changes sample efficiency sharply, and Charton (2024) explains a transformer’s GCD predictions by rules built from divisors of the number base, the kind of surface rule we probe for. Dziri et al. (2023) show how errors compound across the steps of compositional tasks. Filler tokens can replace a chain of thought on some tasks (Pfau et al., 2024); our scrambled control keeps the tokens and removes their order, and on divisibility it removes the gain.

Theory. Parity-like targets give gradient-based learners almost no signal from end labels (Shalev-Shwartz et al., 2017). Sub-task decomposition makes otherwise unlearnable sequence-to-sequence tasks learnable (Wies et al., 2023); CoT raises the expressivity of bounded-depth transformers (Feng et al., 2023; Li et al., 2024; Merrill & Sabharwal, 2024), and next-token predictors trained on CoT data are universal learners (Malach, 2024). Transformers can represent automata through shallow shortcuts (Liu et al., 2023). Abbe et al. (2024) define the globality degree of a target, show that it bounds what transformers learn efficiently as the input grows, and show that an inductive scratchpad breaks that barrier; Kim & Suzuki (2025) prove that teacher-forced intermediate parities make parity learnable. Grokking studies learning of algorithmic tables, such as modular arithmetic, from a fraction of their entries (Power et al., 2022). Section 6 uses these results as intuition only.

Traces and verifiers in post-training. Valmeekam et al. (2026) train transformers from scratch on formally verifiable traces and find the semantics of the trace less important than usually assumed. Process supervision rewards steps rather than outcomes (Lightman et al., 2024), and program-generated logic corpora improve LLM reasoning (Morishita et al., 2024). STaR fine-tunes on self-generated rationales that reach verified answers (Zelikman et al., 2022); RL with verifiable rewards (RLVR) drives recent reasoning models (Lambert et al., 2024; DeepSeek-AI et al., 2025). Fine-tuning on model-generated correct solutions can amplify spurious correlations (Setlur et al., 2024); SFT and RL generalize differently (Chu et al., 2025); and RLVR mostly reweights what the base model already samples (Yue et al., 2025). Our answer arm is an instance of shortcut learning (Geirhos et al., 2020); exact labels do not prevent it.

3 SETUP

Exact verifiers as an instrument. Each task has an exact decision procedure: trial division for primality, the remainder for divisibility, and Z3 (de Moura & Bjørner, 2008) for validity (premises entail C iff premises $\wedge \neg C$ is unsatisfiable). Labels are correct by construction, every arm sees the same prompts and labels, and arms differ only in the completion the model is trained to produce.

Tasks. Table 1 summarizes them. **prime:** is a uniformly drawn 4-digit n prime? The trace performs trial division by the primes up to $\lfloor \sqrt{n} \rfloor$, one “ $n \bmod p = m$.” step per prime, and stops at the first divisor, so a prime needs 11–25 steps and a composite a median of 1. **divd:** is n divisible by d ? The trace performs long division, $r \leftarrow (10r + x_t) \bmod d$ for each digit x_t , so k equals the number of digits; we use $d \in \{2, 3, 7, 11, 13\}$ with 4 digits and $d = 7$ with 6 digits. **valid:** does a propositional conclusion follow from its premises? Arguments instantiate 12 forms (6 valid, 6 fallacies) with random atom letters, and the trace is the Z3 certificate; train and test share forms, so this task is surface-learnable by design (Appendix B). Each test set has 240 items (120 per class), is problem-disjoint from training and identical across arms; training sets are balanced samples of $n \in \{60, 180, 540\}$ (and, for S7 and S9, other n) from a training pool.

Table 1: Tasks. k : steps in the trace (prime: for primes; a composite’s trace stops at its smallest factor). Tokens: mean whitespace tokens per trace. m : supervised transitions per (remainder, digit) table entry at $n = 180$.

task	trace procedure	k	tokens	m
prime	trial division by the primes $\leq \lfloor \sqrt{n} \rfloor$	11–25	71.5	–
div d , 4 digits	long division, $r \leftarrow (10r + x_t) \bmod d$	4	58.0	36.0 ($d=2$) ... 5.5 ($d=13$)
div7, 6 digits	same, $d = 7$	6	76.0	15.4
valid	Z3 countermodel or unsatisfiable core	certificate	37.0	–

Arms. **base:** the model zero-shot. **A:** the completion is “Answer: Yes” or “Answer: No”. **B:** the deterministic trace, a newline, then the answer (Appendix C). **C** (scrambled trace): B’s traces are permuted across training examples, so that trace length cannot signal the label, then token-shuffled within each trace; the answer line keeps the example’s true label (token totals within 7% of B’s). **A-full:** answers on the whole training pool (1,320 examples for prime, 1,600 for valid). **Shuffled labels:** from an earlier study with the same model and LoRA configuration (240 training items). Preregistered additions: **B’** (answer, then trace) on div7, whose answer cannot be computed by its trace; **D** (S8), a fluent, correct long-division trace of a *different* number with the same label; and, on the base model, a zero-shot CoT prompt and 4-shot trace demonstrations. **S** (self-generated, STaR-style; not a decision rule): the base model samples traces, the exact gate keeps those whose answer is correct, and the model is fine-tuned on them.

Training and evaluation. LoRA (Hu et al., 2022) with $r = 8$, $\alpha = 16$, dropout 0.05 on the attention projections of Qwen2.5-1.5B-Instruct (Qwen et al., 2024); AdamW, learning rate 2×10^{-4} , batch size 1, 3 epochs, bf16; loss on completion tokens only. Evaluation is greedy generation, and one extractor serves every arm: the last “Answer: Yes/No”, else the first standalone yes/no. Each run is a fresh process on one consumer GPU and takes 1–17 minutes at $n = 180$.

Statistics, preregistration and provenance. Paired comparisons use the exact McNemar test (McNemar, 1947) on per-item correctness within a seed, with Holm correction (Holm, 1979) across tasks at $\alpha = 0.05$; they treat test items as the unit, so we also report seed SDs and bootstrap intervals over items and seeds. Two preregistrations govern the paper. The first was frozen before the prior grid of 36 runs and fixed four endpoints: B beats A (H1), B beats C (H2), B is more efficient per training token (H3), and on div7 B reaches about 60% with A at chance (H4). The second was committed (b0370f5) at 20:29:47 on 2026-09-25, before any of its runs started, and fixes S1–S6 (Appendix D). Its decision log adds, each before the cells it governs had finished, a clarification of S5’s primary cell (21:26:10), S7 (21:29:52) and S8–S9 (21:38:05). S3 and S6 add little evidence: S3 follows from the last-digit rule, and S6 was effectively settled before it was registered, since A already missed all 15 hard negatives of the prior test set. The surface probe belongs to neither pre-registration. *Provenance.* The prior study’s written verdict (2026-08-04) that traces do not unlock div7 was written from the 60-example trace run alone (50.0%) while the preregistered 180-example cell was still running; that cell finished at 92.5% with no code change.

4 RESULTS

4.1 EXACT ANSWER LABELS TEACH THE SURFACE

Answers track surface probes. We fit simple probes on the same 180 training items as arm A and score them on the test set: logistic regression, a one-hidden-layer MLP and k -nearest neighbours, on 40 one-hot digit-by-position features alone or with engineered divisibility features (last digit even; last digit 0 or 5; digit sum mod 3 and mod 9; alternating digit sum mod 11); for arguments, word and character n -grams with atoms renamed by first appearance. Hyperparameters and the reported probe are chosen by 5-fold cross-validation on the training items; on div2, div3 and div11 the engineered features contain the answer, so there the probe uses one-hot features only. A tracks the probe wherever the probe is fair (Figure 1b): on prime A lies between the one-hot linear probe (79.3%) and the best probe (89.7%), on div7 both are at chance, and on valid both are near ceiling.

Table 2: Accuracy (%) at $n = 180$, Qwen2.5-1.5B-Instruct, 240 test items per task. A: answers; B: trace, then answer; C: scrambled trace; mean over seeds $\pm$ SD across seeds (div7: three seeds; prime, valid: two; other tasks: one so far). B–A: difference in points with a 95% bootstrap interval over items and seeds. max p : the largest exact McNemar p (B vs. A) over seeds; per-seed values are in the text and Appendix A. McNemar treats the 240 items as the unit and so counts items, not training runs, for which the seed SD is the relevant uncertainty.

task	base	probe	A	B	C	B–A [95% CI]	max p
prime	59.6	89.7	82.3 $\pm$ 0.3	57.7 $\pm$ 0.3	52.1 $\pm$ 2.9	-24.6 [-33.3, -15.8]	5.7×10^{-7}
div7	62.5	46.5	50.3 $\pm$ 2.1	88.3 $\pm$ 5.2	50.0 $\pm$ 0.6	+38.1 [+31.0, +43.8]	7×10^{-13}
valid	30.0	100.0	99.6 $\pm$ 0.6	90.0 $\pm$ 14.1	52.5 $\pm$ 2.4	-9.6 [-23.3, +1.7]	(0.50, 7.1×10^{-15})
div3	[TBD]	50.7	50.0	100.0	[TBD]	+50.0 [+43.8, +56.2]	
div11	[TBD]	50.3	50.8	99.2	[TBD]	+48.3 [+42.1, +55.0]	7.2×10^{-34}
div13	[TBD]	53.3	49.6	54.6	[TBD]	+5.0 [-6.2, +15.8]	0.41
div7, 6 digits	[TBD]	48.1	47.9	96.2	[TBD]	+48.3 [+41.7, +55.0]	1.1×10^{-32}

The primality shortcut. A’s answers (recovered from per-item correctness) coincide with “the last digit is 1, 3, 7 or 9” on 99.6% and 100.0% of test items in seeds 0 and 1, and that rule alone scores 82.5%, within a point of A. A has learned divisibility by 2 and 5, the divisors visible in the last digit: it calls 99.6% of primes prime and gets 65.0% of composites right. The rule mislabels every composite ending in 1, 3, 7 or 9, whatever its factors. The test set has 27 such composites with a factor 3 or 7, easy for any sieve, and A gets 0 and 0 of them right (seeds 0 and 1); it also misses all 15 odd composites without a factor ≤ 7 (correct: 0 and 0). On the same test set, “odd” scores 76.2%, “no factor 2 or 3” 86.7% and “no factor ≤ 7 ” 93.8%. The preregistered S6 evaluates the prime adapters on 240 numbers, 120 hard negatives and as many primes, on which the rule calls every item prime: A reaches [TBD]% ([TBD]% on the hard negatives; B: [TBD]%; S6 [TBD]).

More answers do not buy the program. On prime, A reaches 59.6%, 82.3% and 75.0% at $n = 60$, 180 and 540, and 80.8% with the full pool. On div7, A is at chance at every n (50.0%, 50.3% and 50.0%) and learns a seed-dependent constant answer (seed 0: 25.8% correct on multiples of 7, 78.3% on non-multiples; seed 1: 82.5% and 19.2%). It is also below the zero-shot base (62.5%; McNemar $p = 0.047$, 0.0046 and 6.5×10^{-5} for seeds 0–2): the base model answers after long free-form responses (its evaluation on prime takes 38 times as long as A’s), which answer-only training removes. Part of what a trace adds may therefore be test-time computation; B’ and the prompting controls (§4.3) separate the two. With labels permuted across training items, the earlier study’s answer arm sat at 48.5–52.5% on all seven of its tasks (Appendix A), so correct labels are what drive A; on primality, what they buy is the rule.

4.2 WHEN EXACT TRACES TEACH

Divisibility by 7. Over three seeds B reaches 88.3% (92.5, 90.0, 82.5) against A’s 50.3% (52.1, 50.8, 47.9; $p = 4.1 \times 10^{-27}$, 2×10^{-18} , 6.8×10^{-13}), and C stays at 50.0% ($p \leq 2 \times 10^{-20}$ against B). Seeds 1 and 2 are the preregistered replication (S1 passed), and a re-run of seed 0 with the generation-saving worker reproduced 92.5%. The gain holds on both classes (82.2% on multiples, 94.4% on non-multiples). At $n = 60$ the same arm was at 50.0% (one seed).

What the traces get right and wrong (descriptive). In the saved generations, every div7 trace has all 4 steps, digits and carried remainders are copied without error (100.0% and 100.0%), and every error is a wrong lookup of $(10r + x) \bmod 7$. The lookups are right 0.947 of the time; independent errors would leave $0.947^4 = 80.4\%$ of traces error-free, and 82.9% are. When a trace contains an error the answer is still right 56.1% of the time, since a wrong remainder is usually still non-zero. These are measurements of trained models, not predictions.

Divisibility by 13 and 11. At the same length the trace fails for $d = 13$: B 54.6% against A 49.6% ($p = 0.41$, seed 0). Only 2.1% of its traces are correct; the first error comes at step 2 in 188 of 240 items, and 179 of the 235 wrong traces end at $r = 0$, so the model answers “yes” 76% of the time. For $d = 11$ and $d = 3$ the trace works (99.2% and 100.0%, against A at 50.8% and 50.0%, seed

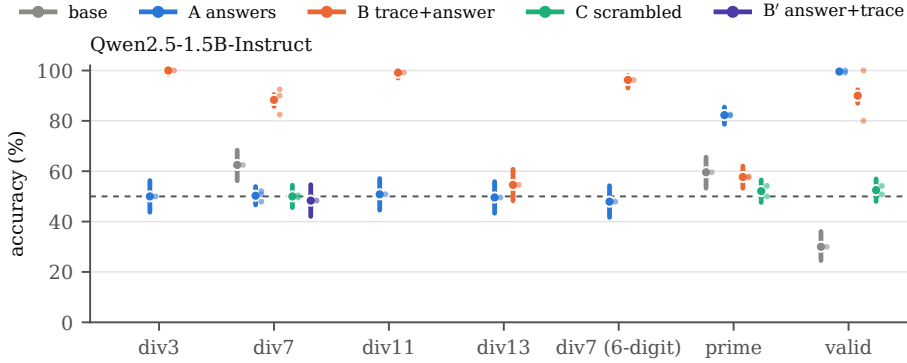


Figure 2: Accuracy by arm and task at $n = 180$, pooled over seeds with 95% Wilson intervals (240 test items per task and seed); small dots are single seeds, one row per model. Dashed line: majority class.

0), with per-step accuracy 0.989 for $d = 11$. Their steps are easier: $10 \equiv 1 \pmod{3}$ and $10 \equiv -1 \pmod{11}$, so the update is $r' = (r + x) \bmod 3$ or $(x - r) \bmod 11$, whereas $d = 7$ and $d = 13$ require a multiplication ($r' = (3r + x) \bmod 7$, $(x - 3r) \bmod 13$). S2 and S3 are in Table 3.

Length (S5): failed as preregistered. The preregistered prediction for 6-digit div7 was $B \approx p^6 = 75.8\%$, with $p = 0.955$ measured on the 4-digit B generations. The primary cell, B trained and tested on 6-digit traces, reached 96.2% (A: 47.9%), outside the 10-point tolerance, so S5 failed. The 6-digit model’s own per-step accuracy is 0.988: p is not a fixed property of the task but depends on the training set. Two forms that allow for a wrong trace ending in a right answer, $(1+p^6)/2 = 87.9\%$ and $p^6 + (1 - p^6)g = 89.4\%$ with g the 4-digit guess rate, were written down after the S5 cell was specified; they are post hoc and do not change the verdict. The secondary cell, the 4-digit adapter on 6-digit inputs, gives [TBD]%.

Transitions per table entry (S7, pending). Because p is measured after training, we preregistered, after S5 failed and before any of its cells ran, a quantity fixed before training: $m = kn/(10d)$, the number of supervised transitions per entry of the (remainder, digit) table. The three cells observed at registration are ordered by m : div13 ($m = 5.5$, $p = 0.566$), div7 (10.3, 0.955) and 6-digit div7 (15.4, 0.988). S7 predicts five further cells and passes if four hold (Table 3). The first to finish, div11 at $m = 6.5$, reached $p = 0.989$, above the predicted interval (0.566, 0.955): prediction (d) failed. Step difficulty varies with d and confounds m , so a test of m must hold d fixed, as predictions (a)–(c) and S9 (div13 at $n = 360$, $m = 11.1$: $B \geq 80\%$, $A \leq 60\%$) do. S7 [TBD]; S9 [TBD]. We do not claim S7.

Primality: the trace hurts. B reaches 57.7% against A’s 82.3% ($p = 5.7 \times 10^{-7}$ and 1.7×10^{-7}), and B vs. C is not significant ($p = 0.22$ and 0.33): the trace’s content gives no detectable benefit over its scrambled tokens. B classifies 19.6% of primes and 95.8% of composites correctly, but scrambled C shows the same split in seed 1 (20/120 primes, 110/120 composites), so the split reflects a learned bias toward “composite” rather than evidence about trace length. At $n = 540$, B reaches 68.8% against A’s 75.0% ($p = 0.19$).

Validity and cost. On valid both formats work: A 99.6%, B 90.0% (100.0% and 80.0% by seed), C 52.5% (Appendix B). Under the Qwen tokenizer, which splits numbers into single digits, B’s training completions contain 14–47 times as many tokens as A’s (33,575 vs. 720 per draw on prime). Of the first preregistration’s endpoints, H1 ($B > A$) held on div7 only, H2 ($B > C$) on div7 and valid but not prime, H3 (efficiency) failed where answers work, and H4 held at $n = 180$ but not $n = 60$.

Table 3: Preregistered tests (Appendix D) with results as they came out. Qwen2.5-1.5B-Instruct, 4 digits and $n = 180$ unless stated; accuracy in %; seed 0 unless stated. For S7 the B column is the per-step accuracy p ; for S8 and arm S it is the accuracy of arms D and S.

	cell	probe	A	B	criterion	met
S1	div7, seeds 1 / 2	46.5	50.8 / 47.9	90.0 / 82.5	$B \geq 80, B-A \geq 20$ pp, $p < 0.01$ in each seed	passed
S2	div3	50.7	50.0	100.0	≥ 2 of the 3 with $B-A$	[TBD]
	div11	50.3	50.8	99.2	≥ 15 pp in both seeds	
	div13	53.3	49.6	54.6		
S3	div2	100.0	[TBD]	[TBD]	$A \geq 95$	[TBD]
S4	div7, 3B model		[TBD]	[TBD]	$B > A$	[TBD]
S5	div7, 6 digits		47.9	96.2	$ B - p^6 \leq 10$ pp; $p^6 = 75.8$	failed
S6	prime $\rightarrow$ hard set		[TBD]	[TBD]	$A < 60$; agrees more with the rule than with the label	[TBD]
S7	(d) div11			0.989	≥ 4 of (a)–(e); (d): $0.566 < p < 0.955$	[TBD] (d failed)
S8	div7, arm D			[TBD]	$D \leq 60$ in both seeds	[TBD]
S9	div13, $n = 360$		[TBD]	[TBD]	$B \geq 80, A \leq 60$	[TBD]
–	div7, arm S			[TBD]	not a decision rule	–

4.3 CONTROLS

Content, grounding and position. C sits at 50.0% on div7 and 52.5% on valid, far below B, although it trains on B’s tokens with the true answers. The preregistered grounding control D (S8) trains on fluent, correct traces of other numbers with the right label; it tests whether the trace must be about the input, and predicts $\leq 60\%$: [TBD]% (S8 [TBD]). B’ puts the answer before the trace, so the procedure cannot compute it; if B’ exceeds A, trace tokens change what is learned, and if it matches A, the gain is test-time computation: 48.3%.

Prompting and self-generated traces. On the base model, a zero-shot CoT prompt gives [TBD]%, 67.1% and [TBD]% on prime, div7 and valid, and 4 trace demonstrations give [TBD]%, [TBD]% and [TBD]%. Arm S, fine-tuned on the base model’s own gate-verified traces, gives [TBD]% on div7.

5 SCALE

Under the same protocol, Qwen2.5-3B-Instruct gives, as A/B: prime [TBD]/[TBD], div7 [TBD]/[TBD], valid [TBD]/[TBD] (S4 [TBD]). Where compute allowed, div7 at 0.5B gives [TBD]/[TBD] and at 7B [TBD]/[TBD]. The surface probe does not depend on the model, so answer-only accuracy should stay near it at every size; how per-step accuracy moves with size at fixed m is open.

6 AN AUTOMATON VIEW (INTUITION)

Remark 1 (intuition, not a theorem). Let $x_1 \cdots x_k$ be the digits of n and $\delta_d(r, a) = (10r + a) \bmod d$. Then $d \mid n$ iff $\delta_d^*(0, x_1 \cdots x_k) = 0$: divisibility by d is recognized by a d -state automaton reading digits. The arm-B trace writes the states $r_1, \dots, r_k$. Under teacher forcing, each supervised state is a function of two tokens already in the context, so n traces are kn samples of one table with $10d$ entries, $m = kn/(10d)$ per entry (720 samples of 70 entries for div7 at $n = 180$). Answer-only supervision gives n samples of the k -fold composition, and for d coprime to 10 the label is nearly independent of every proper subset of the digits: for $d = 7$ and $k = 4$, fixing any three digits leaves $\Pr(7 \mid n)$ between 0.10 and 0.22, against a marginal of 0.14.

This view resembles the globality barrier of Abbe et al. (2024) and the teacher-forced CoT of Kim & Suzuki (2025), but those are statements about learning as the input length grows, and they do not apply literally to fixed 4-digit inputs; we use the analogy as intuition only. The view counts how

often each transition is supervised but not how hard it is to learn: for $d = 3$ and $d = 11$ the transition is an addition or subtraction mod d , for $d = 7$ and $d = 13$ a multiplication, and the results of §4.2 follow the difficulty as much as m . It also suggests why primality fails: a trial-division step is a table indexed by (n, p) with more than 10^5 entries, so $m \ll 1$ at $n = 180$. Whether transformers learn such tables through the kind of shortcut Liu et al. (2023) describe, and how the rate depends on coverage of the table, as in grokking (Power et al., 2022), is what S7 and S9 begin to test. On valid the trace is a certificate whose first clause fixes the answer, so k is effectively 1.

7 DISCUSSION AND LIMITATIONS

What exact labels guarantee. An exact verifier guarantees that no training label is wrong, not that the labels identify the verifier’s function among those a model can fit from n examples. The rule A learned on primality calls every composite ending in 1, 3, 7 or 9 prime; by construction that includes all 996 of the 7,939 4-digit composites without a factor ≤ 7 . A shuffled-label control shows that the gain comes from label correctness but cannot tell the verifier’s function from a surface rule; inputs on which the probe or the rule is wrong can.

Supervision design for verifiable-reward pipelines. (1) Fit a cheap surface probe on the verified data before training; where it is accurate, answer supervision will teach its function, so evaluate on inputs where it is wrong. (2) A correct trace can still fail (div13, primality); count how often each step type is supervised and measure per-step accuracy on held-out items after a small run. (3) Do not read a null from one training size; our own 60-example null on div7 was the edge of a transition.

RLVR: a hypothesis, not a result. *Hypothesis (untested).* With a binary verifier the RLVR reward depends on the final answer only, so RLVR is answer supervision with on-policy sampling; on a surface-learnable task a surface rule already earns most of the reward (the last-digit rule is right on 82.5% of our prime test items), and RLVR should find a shortcut of the same kind, while on div7 it should find the procedure only if the policy’s own samples sometimes execute it, consistent with Yue et al. (2025). We ran no RL. Arm S, the self-generated STaR-style arm, is the nearest experiment we have ([TBD]% on div7).

Limitations. One model family, with 1.5B as the main size, and one LoRA configuration. Binary tasks with 240 test items; the McNemar p -values count items, and with two or three seeds per cell the replication-level evidence is thinner than they suggest (div7 B has seed SD ± 5.2 points, valid B ± 14.1). Several divisor cells have one seed. One trace format per task, written by us. S5 failed; S7’s first predicted cell failed, and its m is confounded with step difficulty. S3 and S6 were effectively settled before registration. The shuffled-label control comes from an earlier study with a related protocol; the probe was defined after the prior grid; the logic task shares 12 forms between training and test. Fine-tuning on answers also removes the base model’s test-time reasoning.

AI USE STATEMENT

We used an AI assistant (Claude, Anthropic) to write and refactor experiment code, to run the experiments and log their outputs under our direction, to compute the statistics, to draft and edit the manuscript, and to check every bibliography entry against arXiv, OpenReview, proceedings and DOI records. The research question, the arms, the preregistered decision rules and the interpretation were reviewed and approved by the authors before the runs they govern. We reviewed all AI-assisted code, text and numbers, verified reported numbers against the result files, and take responsibility for the content of this paper.

REPRODUCIBILITY STATEMENT

Code and data will be released: the dataset generators (with fixed seeds), the training and evaluation worker, the queue runners, the analysis scripts, both preregistration files with their commit history, and every run’s result record, including per-item correctness from which each paired test can be recomputed; an anonymized copy is in the supplementary material. Section 3 and Appendix C give prompts, trace formats and hyperparameters. Every number in the text is a macro regenerated from

the result files (`paper/exp/build_all.py`). Each run takes 1–17 minutes on one consumer GPU.

ETHICS STATEMENT

The study uses synthetic arithmetic and logic data only and involves no human subjects or personal data.

REFERENCES

- Emmanuel Abbe, Samy Bengio, Aryo Lotfi, Colin Sandon, and Omid Saremi. How far can transformers reason? The globality barrier and inductive scratchpad. In *Advances in Neural Information Processing Systems*, 2024.
- François Charton. Learning the greatest common divisor: explaining transformer predictions. In *International Conference on Learning Representations*, 2024.
- Tianzhe Chu, Yuexiang Zhai, Jihan Yang, Shengbang Tong, Saining Xie, Dale Schuurmans, Quoc V. Le, Sergey Levine, and Yi Ma. SFT memorizes, RL generalizes: A comparative study of foundation model post-training. In *International Conference on Machine Learning*, 2025.
- Leonardo de Moura and Nikolaj Bjørner. Z3: An efficient SMT solver. In *Tools and Algorithms for the Construction and Analysis of Systems (TACAS)*, Lecture Notes in Computer Science, pp. 337–340. Springer, 2008. doi: 10.1007/978-3-540-78800-3_24.
- DeepSeek-AI, Daya Guo, Dejian Yang, Haowei Zhang, Junxiao Song, et al. DeepSeek-R1: Incentivizing reasoning capability in LLMs via reinforcement learning. *arXiv preprint arXiv:2501.12948*, 2025.
- Nouha Dziri, Ximing Lu, Melanie Sclar, Xiang Lorraine Li, Liwei Jiang, Bill Yuchen Lin, Peter West, Chandra Bhagavatula, Ronan Le Bras, Jena D. Hwang, Soumya Sanyal, Sean Welleck, Xiang Ren, Allyson Ettinger, Zaid Harchaoui, and Yejin Choi. Faith and fate: Limits of transformers on compositionality. In *Advances in Neural Information Processing Systems*, 2023.
- Guhao Feng, Bohang Zhang, Yuntian Gu, Haotian Ye, Di He, and Liwei Wang. Towards revealing the mystery behind chain of thought: A theoretical perspective. In *Advances in Neural Information Processing Systems*, 2023.
- Robert Geirhos, Jörn-Henrik Jacobsen, Claudio Michaelis, Richard Zemel, Wieland Brendel, Matthias Bethge, and Felix A. Wichmann. Shortcut learning in deep neural networks. *Nature Machine Intelligence*, 2(11):665–673, 2020. doi: 10.1038/s42256-020-00257-z.
- Sture Holm. A simple sequentially rejective multiple test procedure. *Scandinavian Journal of Statistics*, 6(2):65–70, 1979.
- Edward J. Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. LoRA: Low-rank adaptation of large language models. In *International Conference on Learning Representations*, 2022.
- Juno Kim and Taiji Suzuki. Transformers provably solve parity efficiently with chain of thought. In *International Conference on Learning Representations*, 2025.
- Nathan Lambert, Jacob Morrison, Valentina Pyatkin, Shengyi Huang, Hamish Ivison, Faeze Brahman, et al. Tulu 3: Pushing frontiers in open language model post-training. *arXiv preprint arXiv:2411.15124*, 2024.
- Nayoung Lee, Kartik Sreenivasan, Jason D. Lee, Kangwook Lee, and Dimitris Papailiopoulos. Teaching arithmetic to small transformers. In *International Conference on Learning Representations*, 2024.
- Zhiyuan Li, Hong Liu, Denny Zhou, and Tengyu Ma. Chain of thought empowers transformers to solve inherently serial problems. In *International Conference on Learning Representations*, 2024.

- Hunter Lightman, Vineet Kosaraju, Yura Burda, Harri Edwards, Bowen Baker, Teddy Lee, Jan Leike, John Schulman, Ilya Sutskever, and Karl Cobbe. Let’s verify step by step. In *International Conference on Learning Representations*, 2024.
- Bingbin Liu, Jordan T. Ash, Surbhi Goel, Akshay Krishnamurthy, and Cyril Zhang. Transformers learn shortcuts to automata. In *International Conference on Learning Representations*, 2023.
- Eran Malach. Auto-regressive next-token predictors are universal learners. In *International Conference on Machine Learning*, volume 235 of *Proceedings of Machine Learning Research*, pp. 34417–34431, 2024.
- Quinn McNemar. Note on the sampling error of the difference between correlated proportions or percentages. *Psychometrika*, 12(2):153–157, 1947. doi: 10.1007/BF02295996.
- William Merrill and Ashish Sabharwal. The expressive power of transformers with chain of thought. In *International Conference on Learning Representations*, 2024.
- Terufumi Morishita, Gaku Morio, Atsuki Yamaguchi, and Yasuhiro Sogawa. Enhancing reasoning capabilities of LLMs via principled synthetic logic corpus. In *Advances in Neural Information Processing Systems*, 2024.
- Maxwell Nye, Anders Johan Andreassen, Guy Gur-Ari, Henryk Michalewski, Jacob Austin, David Bieber, David Dohan, Aitor Lewkowycz, Maarten Bosma, David Luan, Charles Sutton, and Augustus Odena. Show your work: Scratchpads for intermediate computation with language models. *arXiv preprint arXiv:2112.00114*, 2021.
- Jacob Pfau, William Merrill, and Samuel R. Bowman. Let’s think dot by dot: Hidden computation in transformer language models. *arXiv preprint arXiv:2404.15758*, 2024.
- Alethea Power, Yuri Burda, Harri Edwards, Igor Babuschkin, and Vedant Misra. Grokking: Generalization beyond overfitting on small algorithmic datasets. *arXiv preprint arXiv:2201.02177*, 2022.
- Qwen, An Yang, Baosong Yang, Beichen Zhang, Binyuan Hui, Bo Zheng, Bowen Yu, et al. Qwen2.5 technical report. *arXiv preprint arXiv:2412.15115*, 2024.
- Amrith Setlur, Saurabh Garg, Xinyang Geng, Naman Garg, Virginia Smith, and Aviral Kumar. RL on incorrect synthetic data scales the efficiency of LLM math reasoning by eight-fold. In *Advances in Neural Information Processing Systems*, 2024.
- Shai Shalev-Shwartz, Ohad Shamir, and Shaked Shammah. Failures of gradient-based deep learning. In *International Conference on Machine Learning*, volume 70 of *Proceedings of Machine Learning Research*, pp. 3067–3075, 2017.
- Zayne Sprague, Fangcong Yin, Juan Diego Rodriguez, Dongwei Jiang, Manya Wadhwa, Prasann Singhal, Xinyu Zhao, Xi Ye, Kyle Mahowald, and Greg Durrett. To CoT or not to CoT? Chain-of-thought helps mainly on math and symbolic reasoning. In *International Conference on Learning Representations*, 2025.
- Karthik Valmeekam, Vardhan Palod, Kaya Stechly, Atharva Gundawar, and Subbarao Kambhampati. Beyond semantics: The unreasonable effectiveness of reasonless intermediate tokens. *Transactions on Machine Learning Research*, 2026.
- Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Brian Ichter, Fei Xia, Ed Chi, Quoc Le, and Denny Zhou. Chain-of-thought prompting elicits reasoning in large language models. In *Advances in Neural Information Processing Systems*, 2022.
- Noam Wies, Yoav Levine, and Amnon Shashua. Sub-task decomposition enables learning in sequence to sequence tasks. In *International Conference on Learning Representations*, 2023.
- Yang Yue, Zhiqi Chen, Rui Lu, Andrew Zhao, Zhaokai Wang, Yang Yue, Shiji Song, and Gao Huang. Does reinforcement learning really incentivize reasoning capacity in LLMs beyond the base model? In *Advances in Neural Information Processing Systems*, 2025.
- Eric Zelikman, Yuhuai Wu, Jesse Mu, and Noah D. Goodman. STaR: Bootstrapping reasoning with reasoning. In *Advances in Neural Information Processing Systems*, 2022.

A FULL RESULTS

Table 4 lists every run as logged. “B-ext” uses reasoning traces sampled from a locally served 9B model and kept only when their final answer matched the verifier (STaR-style rejection sampling); only 9 traces passed, so these runs are not comparable to B and are listed for completeness. Train tokens count whitespace tokens in the completions. Table 5 reports the earlier answer-only study with its shuffled-label control.

Table 4: Every run, as logged (240 test items per cell). Cell: the test set, prefixed by the training task when they differ; [cot] and [fewshot4] mark prompting controls on the base model. Train tokens: whitespace tokens in the completions.

model	cell	arm	n	seed	train tokens	accuracy (%)
1.5B	prime	base	0	0	0	59.6
1.5B	prime	A	60	0	120	59.6
1.5B	prime	A	180	0	360	82.1
1.5B	prime	A	180	1	360	82.5
1.5B	prime	A	540	0	1,080	75.0
1.5B	prime	B	60	0	4,355	57.1
1.5B	prime	B	180	0	13,370	57.9
1.5B	prime	B	180	1	13,260	57.5
1.5B	prime	B	540	0	40,095	68.8
1.5B	prime	C	180	0	12,561	50.0
1.5B	prime	C	180	1	13,120	54.2
1.5B	prime	A-full	1320	0	2,640	80.8
1.5B	prime	B-ext	9	0	896	62.1
1.5B	prime	B-ext	9	1	974	60.0
1.5B	div7	base	0	0	0	62.5
1.5B	div7	A	60	0	120	50.0
1.5B	div7	A	180	0	360	52.1
1.5B	div7	A	180	1	360	50.8
1.5B	div7	A	180	2	360	47.9
1.5B	div7	A	540	0	1,080	50.0
1.5B	div7	B	60	0	3,600	50.0
1.5B	div7	B	180	0	10,800	92.5
1.5B	div7	B	180	1	10,800	90.0
1.5B	div7	B	180	2	10,800	82.5
1.5B	div7	C	180	0	10,784	49.6
1.5B	div7	C	180	1	10,824	50.4
1.5B	div7	B'	180	0	10,800	48.3
1.5B	div7[cot]	base	0	0	0	67.1
1.5B	valid	base	0	0	0	30.0
1.5B	valid	A	60	0	120	97.5
1.5B	valid	A	180	0	360	99.2
1.5B	valid	A	180	1	360	100.0
1.5B	valid	A	540	0	1,080	100.0
1.5B	valid	B	60	0	2,343	87.5
1.5B	valid	B	180	0	7,001	100.0
1.5B	valid	B	180	1	7,004	80.0
1.5B	valid	B	540	0	21,084	100.0
1.5B	valid	C	180	0	7,092	54.2
1.5B	valid	C	180	1	7,090	50.8
1.5B	valid	A-full	1600	0	3,200	100.0
1.5B	div3	A	180	0	360	50.0
1.5B	div3	B	180	0	10,800	100.0
1.5B	div11	A	180	0	360	50.8
1.5B	div11	B	180	0	10,800	99.2
1.5B	div13	A	180	0	360	49.6
1.5B	div13	B	180	0	10,800	54.6
1.5B	div7_6d	A	180	0	360	47.9

model	cell	arm	n	seed	train tokens	accuracy (%)
1.5B	div7_6d	B	180	0	14,040	96.2

Table 5: Earlier answer-only study (same model and LoRA configuration; 240 training items, answer-only prompts, 3–4-digit numbers for arithmetic; mean of two seeds). Correct labels vs. labels permuted across training items.

task	base	correct labels	shuffled labels
divisible by 3	55.0	67.3	49.6
divisible by 7	53.3	51.0	52.5
divisible by 13	54.2	50.0	50.0
perfect square	51.2	83.3	51.5
prime	50.0	82.9	50.0
argument validity	66.7	99.4	50.0
joint consistency (SAT)	50.0	99.8	48.5

B THE LOGIC TASK

Arguments instantiate 6 valid forms (modus ponens, modus tollens, hypothetical syllogism, disjunctive syllogism, constructive dilemma, contraposition) and 6 invalid ones (affirming the consequent, denying the antecedent, affirming a disjunct, a broken chain, the converse, and “or” to “and”) with distinct random atom letters, rendered in English (“if A then B ”, “not A ”, “ A or B ”). The schema proposes an argument and Z3 decides it: the argument is labeled valid iff the premises together with the negated conclusion are unsatisfiable. For a valid argument the trace names the unsatisfiable core; for an invalid one it gives the satisfying assignment as a countermodel. Training and test share the 12 forms and never an instance, so a model can reach high accuracy by recognizing the form, which the atom-renamed n -gram probe does (100.0%). The task therefore checks that answers work when the surface determines the label; it is not a test of logical reasoning. Its results: base 30.0%, A 99.6% (99.2 and 100.0), B 90.0% (100.0 and 80.0), C 52.5% (54.2 and 50.8); at $n = 60$, A 97.5% and B 87.5%; at $n = 540$, both 100.0%.

C TRACE FORMATS

All arms share the prompt, which ends “End your reply with ‘Answer: Yes’ or ‘Answer: No’.” Examples of training completions, verbatim:

div7, B (prompt: “Is 1617 divisible by 7?”):

```
Process the digits of 1617 keeping a running remainder mod
7.  r = (10*0 + 1) mod 7 = 1.  r = (10*1 + 6) mod 7 = 2.  r
= (10*2 + 1) mod 7 = 0.  r = (10*0 + 7) mod 7 = 0.  Final
remainder 0, so 1617 is divisible by 7.
Answer:  Yes
```

div7, C (a scrambled trace from another example; the answer line keeps this example’s label):

```
running mod r = so = 4067 Process by 7.  7 a = 0.  0) 0.  7
mod (10*5 = r 7 the = = keeping + r is mod mod 4) (10*0 + of
4067 r + 7) 5.  = + digits remainder 7.  (10*0 = 7 0,
divisible Final 6) 4.  (10*4 mod remainder
Answer:  Yes
```

prime, B, a composite (“Is 4143 a prime number?”):

```
Check prime divisors up to isqrt(4143) = 64.  4143 mod 2 =
1.  4143 mod 3 = 0.  3 divides 4143, so 4143 is composite.
Answer:  No
```

prime, B, a prime (“Is 1493 a prime number?”):

Check prime divisors up to $\text{isqrt}(1493) = 38$. $1493 \bmod 2 = 1$. $1493 \bmod 3 = 2$. $1493 \bmod 5 = 3$. $1493 \bmod 7 = 2$. $1493 \bmod 11 = 8$. $1493 \bmod 13 = 11$. $1493 \bmod 17 = 14$. $1493 \bmod 19 = 11$. $1493 \bmod 23 = 21$. $1493 \bmod 29 = 14$. $1493 \bmod 31 = 5$. $1493 \bmod 37 = 13$. No prime divisor up to 38, so 1493 is prime.
Answer: Yes

valid, B, invalid (“Premises: (1) if H then D; (2) D. Conclusion: H.”):

Assume every premise is true and the conclusion false. That is satisfiable: set $D=\text{true}$, $H=\text{false}$. Under this assignment all premises hold while the conclusion is false, a countermodel, so the argument is invalid.
Answer: No

valid, B, valid (“Premises: (1) if S then R; (2) S. Conclusion: R.”):

Assume every premise is true and the conclusion false, i.e. $\sim(R)$ holds. Together these force a contradiction: $\{S \rightarrow R, S, \sim(R)\}$ cannot all hold at once. Since assuming the conclusion false is impossible, the conclusion follows.
Answer: Yes

A (every task): Answer: Yes or Answer: No.

D PREREGISTRATION TEXT

First preregistration (prior grid). Frozen before any run of the prior grid and never edited; results were appended below it. It fixed: arms A, B, C, an answers-only arm at larger scale, arms using model-generated traces, and a conditional wrong-trace arm; Qwen2.5-1.5B-Instruct with the LoRA configuration of §3; tasks prime, valid and div7, all reported; 240 problem-disjoint test items per task (120 per class), identical across arms, graded by one parser on the final answer. Endpoints: H1, $B > A$ at $n = 180$ per task, paired McNemar, Holm across the three tasks; H2, $B > C$ at $n = 180$; H3, accuracy at matched training tokens; H4, on div7 B reaches about 60% with A at chance. The full text is in the supplementary material.

Second preregistration and its decision log. Reproduced from PREDICTIONS.md at the time of writing; Markdown markup converted, text unchanged except that the location in each time stamp is redacted. The header’s time was first written as an estimate (“~20:35”) and later replaced by the commit time; the decision log notes this.

Preregistered predictions and decision rules (frozen before any run listed here). Committed 2026-09-25 20:29:47 [location redacted] (commit b0370f5), before any of the runs below were started. The commit hash of this file is the preregistration. Earlier results (results/THINKING_VS_DATA.md, frozen 2026-08-04, and its later staged results block) are prior data, reported as such.

Provenance of the prior div7 result (stated in the paper). The 2026-08-04 verdict “H4 (a procedure trace unlocks div7): NO” was written from the $n = 60$ trace run alone ($B@60 = 50.0\%$). The preregistered main cell, $B@180$ seed 0, finished afterwards at 92.5% with the same worker (no code change). The paper reports the premature verdict and the later result.

Claim under test. Exact labels teach shortcuts; exact traces teach short programs. With labels from an exact verifier: answer-only supervision (arm A) recovers roughly what a simple probe on the answer surface recovers; trace supervision (arm B: procedure trace, then answer) succeeds when the per-step trace accuracy p , compounded over k steps, stays high (predicted accuracy about p^k), and fails when the procedure is long.

Decision rules. S1 (go/no-go, before any other new run): div7 $B@180$ seeds 1 and 2 each reach $\geq 80\%$ accuracy, with $B - A \geq 20$ percentage points and McNemar $p < 0.01$ against A of the same seed. If S1 fails, the paper is not submitted in this form. **S2:** at least

2 of {div3, div11, div13} (4-digit numbers) give $B - A \geq 15$ pp with the same sign in both seeds. **S3:** div2 $A \geq 95\%$ (the answer is surface-learnable from the last digit). **S4:** the div7 sign ($B > A$) holds for Qwen2.5-3B-Instruct. **S5 (falsifiable length test):** on 6-digit div7, B's accuracy is within 10 pp of p^6 , where p is B's per-step accuracy measured on 4-digit div7 generations. **S6 (shortcut):** A on prime agrees more with the rule "odd and no factor ≤ 7 " than with the true label on hard negatives (odd composites with no factor ≤ 7), where A's accuracy drops below 60%.

Definitions: a "win" is a difference of ≥ 10 pp with both seeds agreeing in sign; "neither" is both arms at or below majority class + 8 pp. McNemar tests per seed, Holm correction across tasks within each hypothesis family, $\alpha = 0.05$. Every cell is reported, including those that go against the claim.

Arms. base (zero-shot), A (answers), B (trace then answer), C (length-matched scrambled trace), B' (answer then trace, div7 only), plus eval-only controls: zero-shot chain-of-thought prompt and 4-shot trace demonstrations on the base model. Model: Qwen2.5-1.5B-Instruct (laptop), Qwen2.5-3B-Instruct and others where the RTX 5090 allows. LoRA $r = 8$, $\alpha = 16$, 3 epochs, $\text{lr} = 2e-4$, bf16, greedy decoding, $n = 180$ training examples, 240 problem-disjoint test items.

Decision log.

2026-09-25 21:09:47 (commit fcaa05d) — **S1 PASS.** div7 B@180 seed 1 = 90.0% vs A 50.8% (+39.2 pp, McNemar $p = 2e-18$); seed 2 = 82.5% vs A 47.9% (+34.6 pp, $p = 6.8e-13$). Runs in results/thinking_vs_data/runs_L1.jsonl, produced by the unchanged worker experiments/exp_thinking_ft_worker.py. The paper proceeds; the remaining tests run on experiments/exp_worker_v2.py.

2026-09-25 21:26:10 (commit 27ba807; the div7_6d B seed-0 generation file was written at 21:27:30) — **S5 clarification, recorded before either S5 cell finished** (div7_6d B seed 0 was mid-training; the 4-digit-adaptor transfer eval had not started). Primary S5 cell = arm B trained and tested on the 6-digit task (div7_6d), seeds 0 and 1, compared with p^6 where p = per-step accuracy of 4-digit div7 B generations (seed 0: $p = 0.955$, $p^6 = 0.758$). Secondary = the 4-digit B adaptor evaluated on 6-digit inputs. Also reported, labelled post hoc: the fraction of fully correct 6-digit traces vs p^6 , and answer accuracy vs $p^6 + (1 - p^6) * g$, where g is the answer-correct rate when the trace is wrong (4-digit: $g = 0.561$), since a wrong trace still yields the right yes/no answer about half the time.

2026-09-25 21:29:52 (commit 59e3989) — **S5 (primary) FAILED as preregistered:** 6-digit div7 B seed 0 = 96.2% vs $p^6 = 75.8\%$ (+20.4 pp > 10 pp tolerance). Its own per-step accuracy is 98.8% (vs 95.5% for the 4-digit model), so p is not a fixed property that transfers between training sets.

2026-09-25 21:29:52 (commit 59e3989; div11 B seed 0 started after 21:30:58) — **S7 (new hypothesis, preregistered before any of its cells ran):** per-step trace accuracy p is set by the number of supervised transitions per entry of the (remainder, digit) table, $m = k * n / (10 * d)$ (k steps per trace, n training examples, $10d$ table entries). Observed so far: div7 $n=180$ $m=10.3 \rightarrow p=0.955$; div7_6d $n=180$ $m=15.4 \rightarrow p=0.988$; div13 $n=180$ $m=5.5 \rightarrow p=0.566$. Predictions: (a) div13 B $n=540$ seed 0 ($m=16.6$): $p \geq 0.95$ and answer accuracy $\geq 85\%$; (b) div7 B $n=270$ seed 0 (4-digit, $m=15.4$): $p \geq 0.97$; (c) div7 B $n=90$ seed 0 ($m=5.1$): $p \leq 0.80$ and answer accuracy $\leq 75\%$; (d) div11 B $n=180$ ($m=6.5$, not yet run): $0.566 < p < 0.955$; (e) div3 B $n=180$ ($m=24$) and div2 B $n=180$ ($m=36$): $p \geq 0.97$. S7 passes if at least 4 of (a)-(e) hold; every cell is reported either way.

Note (21:35): the clock times in the entries above were first written as estimates; they have been replaced by the commit timestamps, which are the authoritative record.

2026-09-25 21:38:05 (commit 1163562) **S8 and S9, preregistered before their cells ran** (div11 B seed 0 = 99.2% had finished; none of the cells below had started): S8 (grounding control): div7 arm D (a fluent, correct long-division trace of a DIFFERENT number with the same label) at $n = 180$, seeds 0 and 1: accuracy $\leq 60\%$ in both seeds, i.e. within 10 pp of A, far below B. S9 (dose, step difficulty fixed): div13 B at $n = 360$, seed 0 ($m = 11.1$ transitions per table entry, close to div7's 10.3 at $n = 180$): accuracy $\geq 80\%$; div13 A at $n = 360$ seed 0 stays $\leq 60\%$.

E THE COMPOUNDING APPROXIMATION (POST HOC)

This derivation is post hoc: it was not part of the preregistered S5 rule, and it does not change S5's verdict. Suppose each emitted state is correct with probability p , independently, and that after the first error the emitted final state is uniform on $\mathbb{Z}_d$. With probability p^k the trace is error-free and the answer is right. Otherwise the answer is right with probability $1/d$ on a multiple of d and $(d-1)/d$

otherwise, so on a balanced test set the accuracy is $p^k + \frac{1}{2}(1 - p^k) [\frac{1}{d} + \frac{d-1}{d}] = (1 + p^k)/2$. Both assumptions can fail: errors are correlated across steps (a model that miscomputes one table entry does so whenever the entry recurs), and a wrong state need not be uniform (on div13, wrong traces end at $r = 0$ in 179 of 235 cases). The positive-class accuracy, $p^k + (1 - p^k)/d$, is the cleaner check, because a model that answers “no” by default scores well on non-multiples: on 6-digit div7 it is 94.2%.